



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

Google云计算原理与应用 (三)

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.5 分布式存储系统Megastore

互联网的迅速发展带来了新的数据应用场景，和传统的数据存储有别的是，互联网上的应用对于数据的可用性和系统的扩展性具有很高的要求。一般的互联网应用都要求能够做到7天x24小时的不间断服务，达不到的话则会带来较差的用户体验。

热门的应用往往会在短时间内经历急剧的用户数量增长，这就要求系统具有良好的可扩展性。在互联网的应用中，为了达到好的可扩展性，常常会采用NoSQL 存储方式。

但是从应用程序的构建方面来看，传统的关系型数据库又有着NoSQL所不具备的优势。Google设计和构建了用于互联网中交互式服务的分布式存储系统Megastore，该系统成功的将关系型数据库和NoSQL的特点与优势进行了融合。本节将向大家介绍该系统，着重突出 Megastore设计与构建过程中的核心思想想和技术。

2.5 分布式存储系统Megastore

- ▶ 2.5.1 设计目标及方案选择
- 2.5.2 Megastore数据模型
- 2.5.3 Megastore中的事务及并发控制
- 2.5.4 Megastore基本架构
- 2.5.5 核心技术——复制
- 2.5.6 产品性能及控制措施

2.5 分布式存储系统Megastore

● 设计目标及方案选择

设计目标

设计一种介于传统的关系型数据库和NoSQL之间的存储技术，尽可能达到高可用性和高可扩展性的统一。

方法一

针对可用性的要求，实现了一个同步的、容错的、适合远距离传输的复制机制。

方法二

针对可扩展性的要求，将整个大的数据分割成很多小的数据分区，每个数据分区连同它自身的日志存放在NoSQL数据库中，具体来说就是存放在Bigtable中。

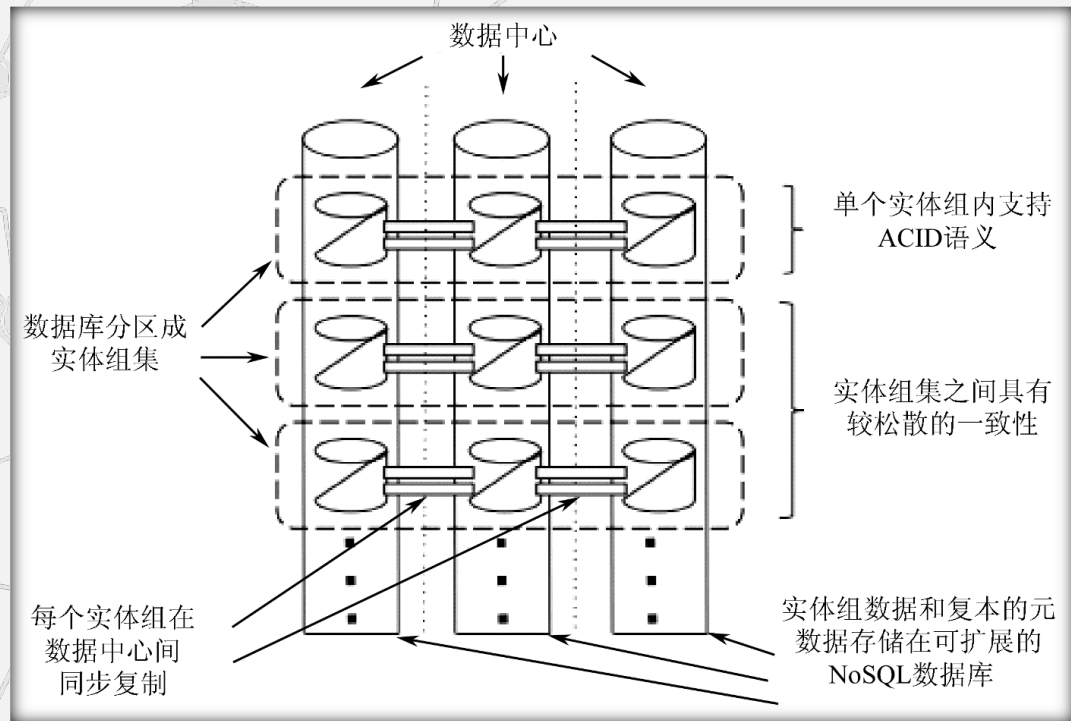
2.5 分布式存储系统Megastore

● 数据的分区和复制

在Megastore中，这些小的数据分区被称为实体组集（Entity Groups）。

每个实体组集包含若干的实体组（Entity Group，相当于分区中表的概念）。

一个实体组中包含很多的实体（Entity，相当于表中记录的概念）。



2.5 分布式存储系统Megastore

2.5.1 设计目标及方案选择

► 2.5.2 Megastore数据模型

2.5.3 Megastore中的事务及并发控制

2.5.4 Megastore基本架构

2.5.5 核心技术——复制

2.5.6 产品性能及控制措施

2.5 分布式存储系统Megastore

● 传统的关系型数据库不合适的三个原因

传统的关系型数据库是通过连接（Join）来满足用户的需求的，但是就Megastore而言，这种数据模型是不合适的，主要有以下三个原因：

原因 1

对于高负载的交互式应用来说，可预期的性能提升要比使用一种代价高昂的查询语言所带来的好处多

原因 2

Megastore所面对的应用是读远多于写，因此好的选择是将读操作所需要做的工作尽可能地转移到写操作上

原因 3

在Bigtable这样的键/值存储系统中存储和查询级联数据（Hierarchical Data）是很方便的



Megastore数据模型 怎么设计?

2.5 分布式存储系统Megastore

- 细粒度控制的数据模型和模式语言



Google团队设计的
Megastore
数据模型

同关系型数据库一样，Megastore的数据模型是在模式（schema）中定义的且是强类型的（strongly typed）

每个模式都由一系列的表（tables）构成，表又包含有一系列的实体（entities），每实体中包含一系列属性（properties）

属性是命名的且具有类型，这些类型包括字符型（strings）、数字类型（numbers）或者Google的Protocol Buffers。

2.5 分布式存储系统Megastore

● 照片共享服务数据模型实例

```
CREATE SCHEMA PhotoApp;

CREATE TABLE User {
  required int64 user_id;
  required string name;
} PRIMARY KEY(user_id), ENTITY GROUP ROOT;

CREATE TABLE Photo {
  required int64 user_id;
  required int32 photo_id;
  required int64 time;
  required string full_url;
  optional string thumbnail_url;
  repeated string tag;
} PRIMARY KEY(user_id, photo_id),
  IN TABLE User,
  ENTITY GROUP KEY(user_id) REFERENCES User;

CREATE LOCAL INDEX PhotosByTime
  ON Photo(user_id, time);

CREATE GLOBAL INDEX PhotosByTag
  ON Photo(tag) STORING (thumbnail_url);
```

- 表Photo就是一个子表，因为它声明了一个外键
- User则是一个根表
- 一个Megastore实例中可以有若干个不同的根表，表示不同类型的实体组集
- 三种不同属性设置，既有必须的（如user_id），也有可选的（如thumbnail_url）
- Photo中的可重复类型的tag属性

2.5 分布式存储系统Megastore

- Megastore索引

主要
两类

局部
索引

定义在单个实体组中，作用域仅限于单个实体组（如PhotosByTime）

全局
索引

可以横跨多个实体组集进行数据读取操作（如PhotosByTag）

额外
索引

STORING子句
(STORING Clause)

可重复的索引
(Repeated Indexes)

内联索引
(Inline Indexes)

2.5 分布式存储系统Megastore

- Bigtable中存储情况

行键 (Row Key)	User.name	Photo.time	Photo.tag	Photo._url
101	John			
101,500		12:30:01	Dinner, Paris	...
101,502		12:15:22	Betty, Paris	...
102	Mary			

Bigtable的列名实际上是表名和属性名结合在一起得到，不同表中实体可存储在同一个Bigtable行中

2.5 分布式存储系统Megastore

2.5.1 设计目标及方案选择

2.5.2 Megastore数据模型

► 2.5.3 Megastore中的事务及并发控制

2.5.4 Megastore基本架构

2.5.5 核心技术——复制

2.5.6 产品性能及控制措施

2.5 分布式存储系统Megastore

- Megastore提供的三种读

current

总是在单个实体组中完成

snapshot

总是在单个实体组中完成

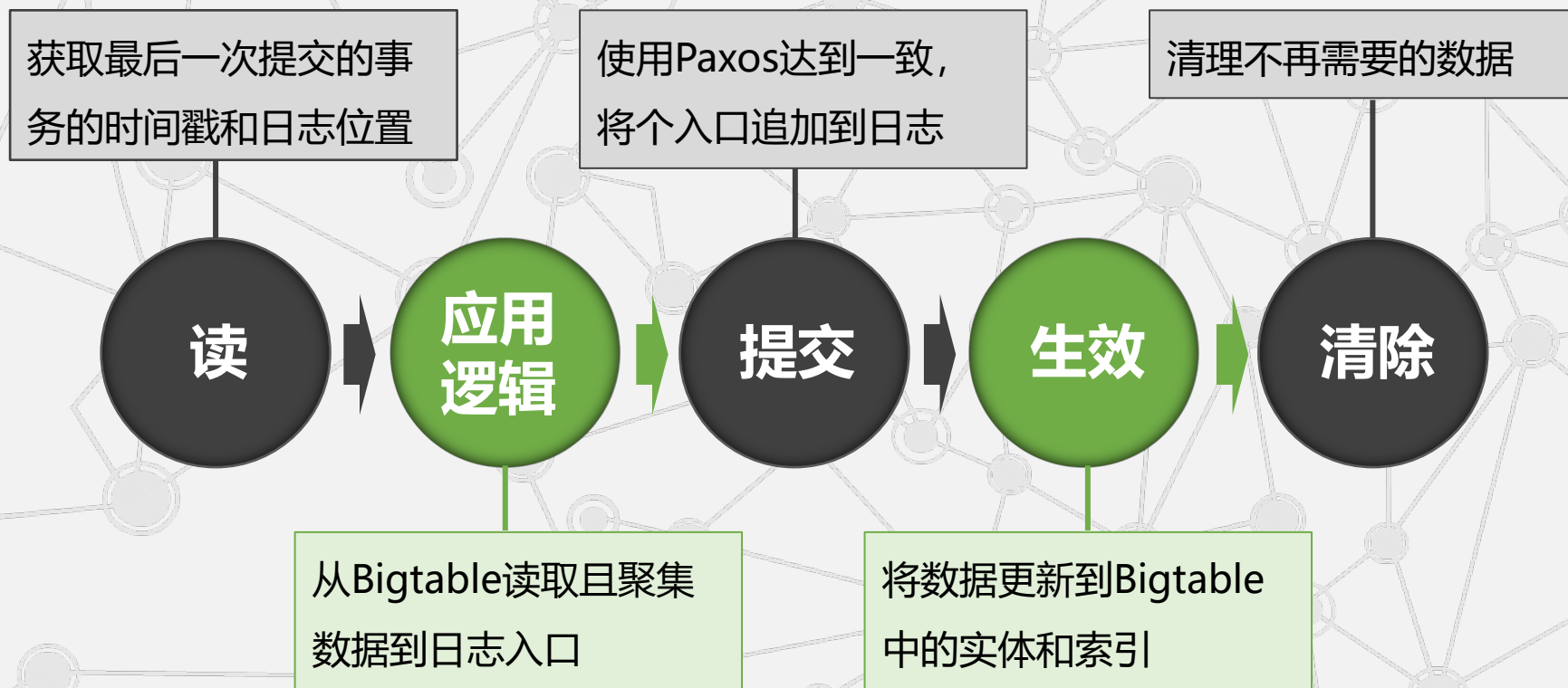
系统取出已知的最后一个完整提交的事务的时间戳，接着从这个位置读数据

inconsistent

忽略日志的状态直接读取最新的值

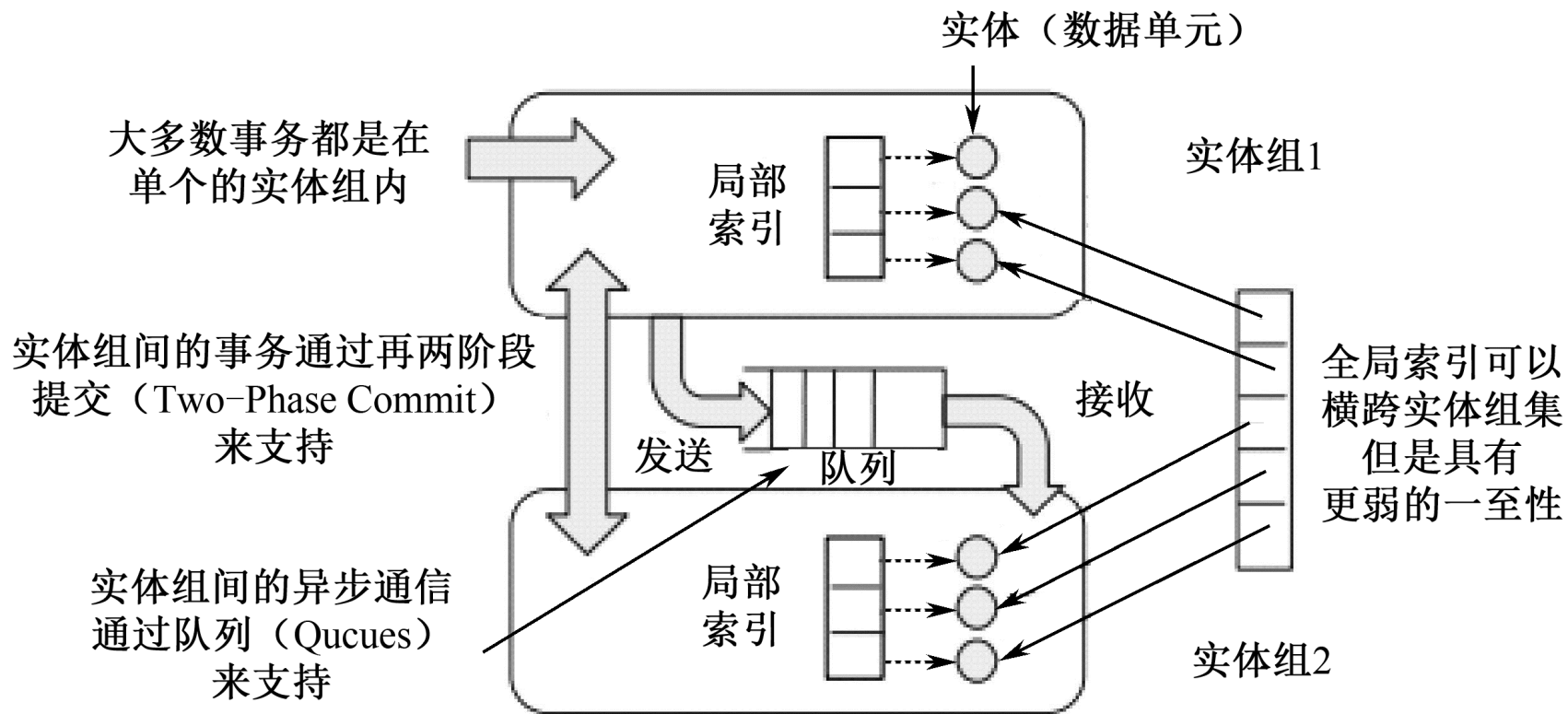
2.5 分布式存储系统Megastore

完整的事务周期



2.5 分布式存储系统Megastore

• Megastore中的事务机制



2.5 分布式存储系统Megastore

Megastore中的消息能够横跨实体本组，在一个事务中分批执行多个更新或者延缓作业(DeferWork)。在单个实体组上执行的事务除了更新它自己的实体外，还能够发送或收到多个信息。每个消息都有一个发关和接收的实体组;如果这两个实体组是不同的，那么传输将会是异步的。虽然这种消息队列机制在关系型数据库中已经有了很长的应用历史，Megastore实现的这种消息机制的最大特点在于其规模:声明一个队列后可以在其他所有的实体组上创建一个收件箱。

除了队列机制之外，Megastore还支持两阶段提交(Two-phase Commit)。但是这会产生比较高的延迟并且增加了竞争的风风险，一般情况下不鼓励使用。

2.5 分布式存储系统Megastore

2.5.1 设计目标及方案选择

2.5.2 Megastore数据模型

2.5.3 Megastore中的事务及并发控制

► 2.5.4 Megastore基本架构

2.5.5 核心技术——复制

2.5.6 产品性能及控制措施

2.5 分布式存储系统Megastore

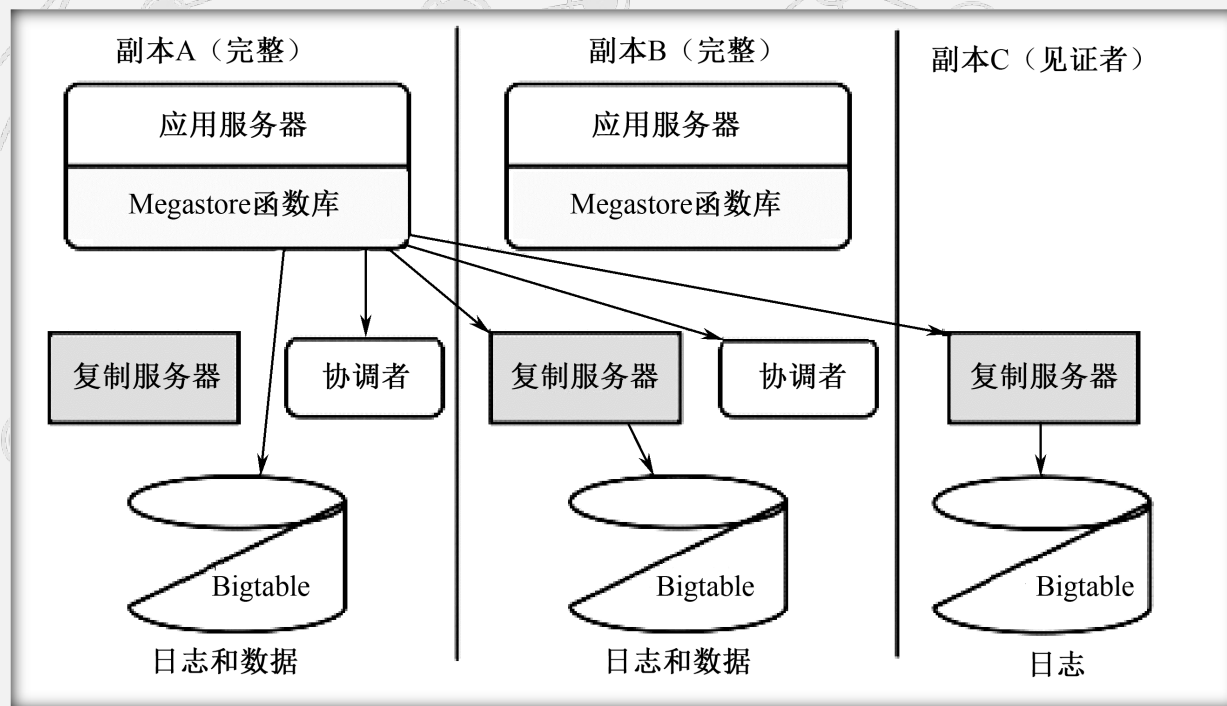
• Megastore基本架构

在Megastore中共有**三种副本**

完整副本
(Full Replica)

见证者副本
(Witness Replica)

只读副本
(Read-only Replica)



2.5 分布式存储系统Megastore

- 快速读与快速写

快速读

利用本地读取实现快速读，带来更好的用户体验及更低的延迟

关键是保证选择的副本上数据是最新的

协调者是一个服务，该服务分布在每个副本的数据中心里面。
它的主要作用就是跟踪一个实体组集合

协调者的状态是由写算法来保证

2.5 分布式存储系统Megastore

- 快速读与快速写

快速写

如果一次写成功，那么下一次写的时候就跳过准备过程，直接进入接受阶段

Megastore没有使用专门的主服务器，而是使用leaders

leader主要是来裁决哪个写入的值可以获取0号提议

客户端、网络及Bigtable的故障都会导致一个写操作处于不确定的状态

2.5 分布式存储系统Megastore

2.5.1 设计目标及方案选择

2.5.2 Megastore数据模型

2.5.3 Megastore中的事务及并发控制

2.5.4 Megastore基本架构

► 2.5.5 核心技术——复制

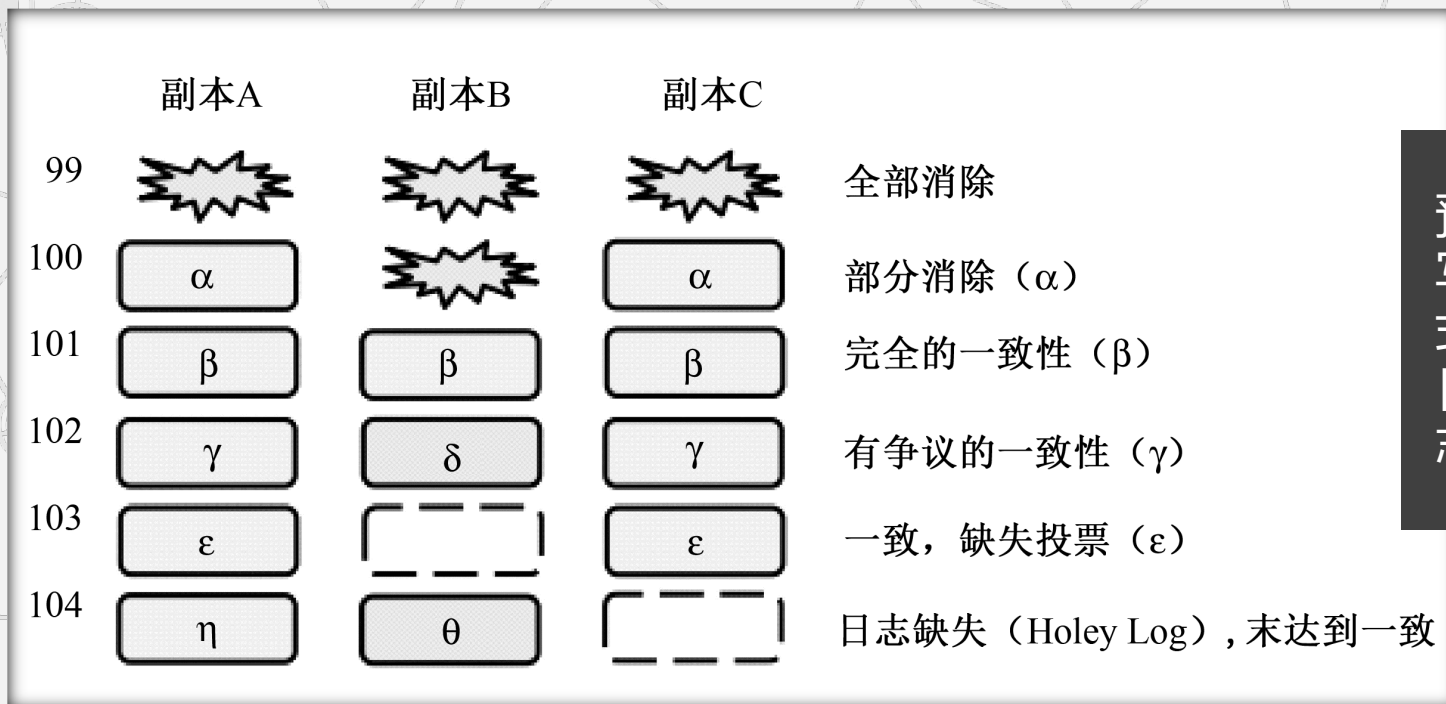
2.5.6 产品性能及控制措施

2.5 分布式存储系统Megastore

● 复制的日志

每个副本都存有记录所有更新的数据。

Megastore允许副本不按顺序接受日志，这些日志将独立的存储在Bigtable中。



预写式日志

2.5 分布式存储系统Megastore

当日志有不完整的前缀时我们就称一个日志副本有“缺失”(Holes)。

0~99的日志位置已经被全部清除，100的日志位置被部分清除，因为每个副本都会被通知到其他副本已经不再需要这个日志。

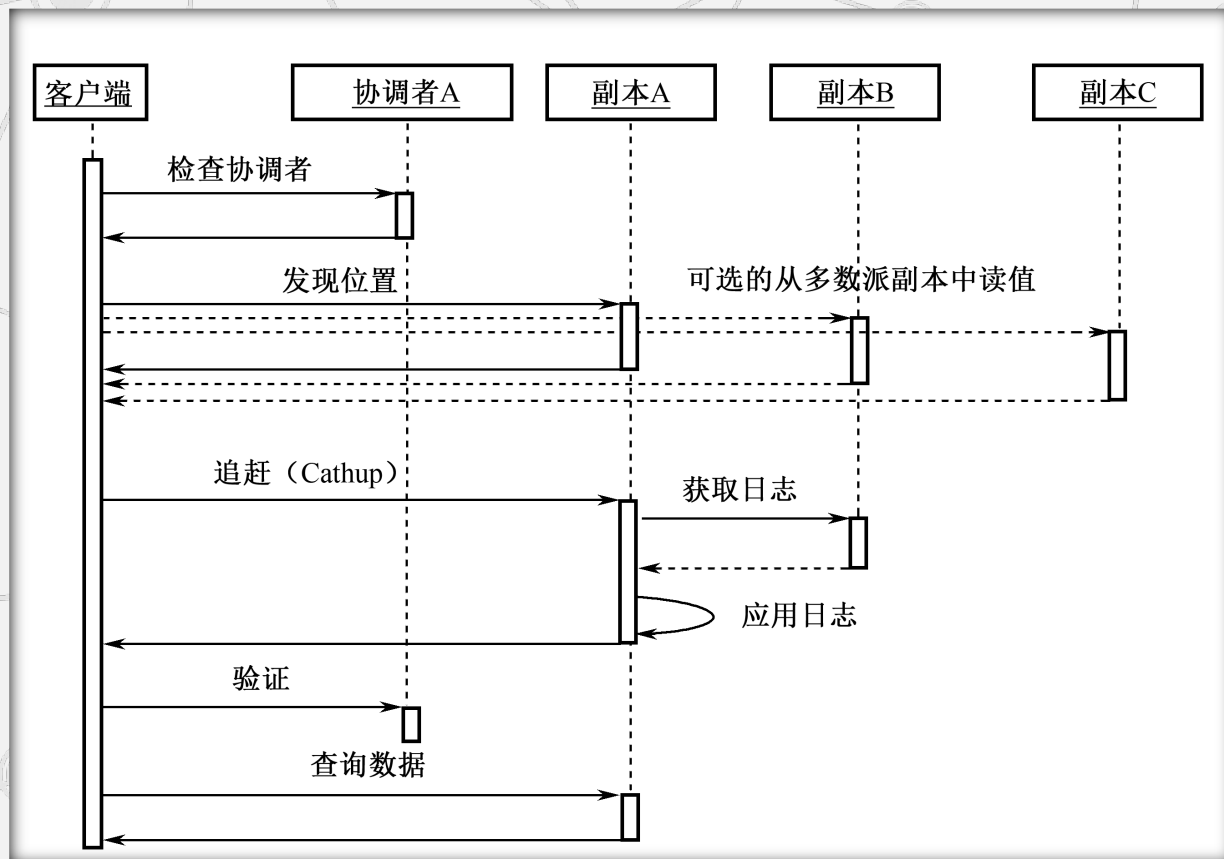
101 的日志位置被全部副本接受。102的日志位置被 γ 获得，这是一种有争议的一致性。

103的日志位置被副本A和副本C接受，副本B则留下了一个“缺失”。

104的日志位置则未达到一致性，因为副本A和副本B存在争议。

2.5 分布式存储系统Megastore

● 数据读取



本地查询

发现位置

追赶

验证

查询数据

2.5 分布式存储系统Megastore

1)本地查询(QueryLocal)

查询本地副本的协调者来决定这个实体组上数据是否已经是最新的。

2)发现位置(FindPosition)

确定一个最高的已经提交的日志位置，选择一个已经在该位置上生效的副本。

(1)本地读取(Local Read):

如果本地查询确定当前的本地副本已经是最新的，则从副本中的最高日志位置和时间戳读取数据。这实际上就是前面提到的快速读。

(2)多数派读取(MajorityRead):

如果本地副本不是最新的(或者本地查询或本地读取超时)，从一个副本的多数派中发现最大的日志位置，然后从中选取一个读取。选择一个响应最快或者最新的副本，并不一定就是本地副本。

2.5 分布式存储系统Megastore

3)追赶

一旦某个副本被选中，就采取如下方式使其追赶到已知的最大日志位置处。

(1)对于所选副本中所有不知道共识值(Consensus Value)的日志位置，从其他的副本中读取值。

对于任意的没有任何可用的已提交的值的日志位置，将会利用Paxos算法发起一次无操作的写。

Paxos将会促使绝大多数副本达成一个共识值--可能是无操作的写也可能是以前的一次写操作。

(2)接下来就所有未生效的日志位置生效成上面达成的共识值，以此来达到一种分布式一致状态。

2.5 分布式存储系统Megastore

4)验证(Validate)

如果本地副本被选中且数据不是最新，发送一个验证消息到协调者断定(entity group, replica)对((entity group, replica) pair)能够反馈所有提交的写操作。

无须等待回应，如果请求失败，下一个读操作会重试。

5)查询数据(QueryData)

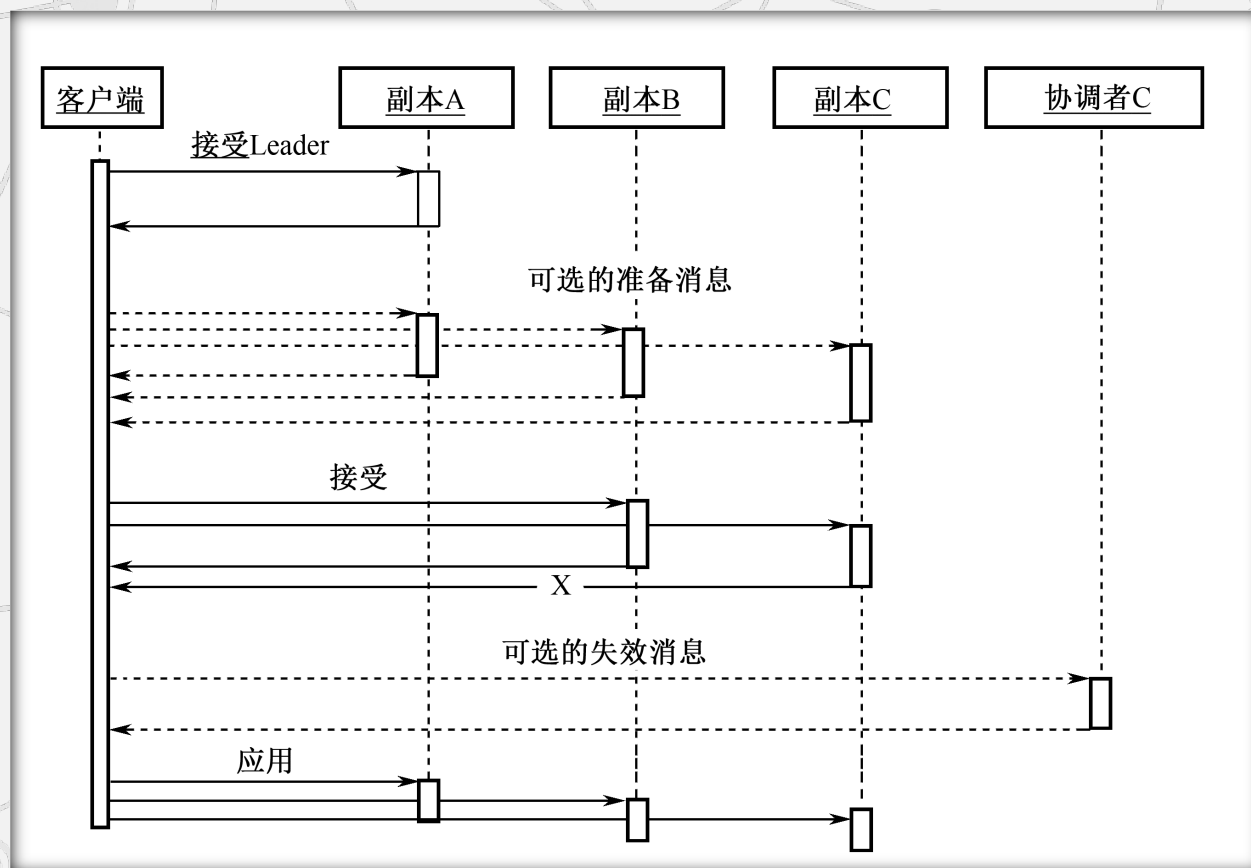
在所选的副本中利用日志位置的时间戳读取数据。如果所选的副本不可用了，重新选中一个替代副本，执行追赶操作，然后从中读取数据。

单个的较大查询结果可能是从多个副本中汇聚而来。

需要指出的是，本地查询和本地读取是并行执行的。

2.5 分布式存储系统Megastore

● 数据写入



接受leader

准备

接受

失效

生效

2.5 分布式存储系统Megastore

- 协调者的可用性

协调者在系统中是比较重要的——协调者的进程运行在每个数据中心。每次的写操作中都要涉及协调者，因此协调者的故障将会导致系统的不可用

Megastore使用了Chubby锁服务，为了处理请求，一个协调者必须持有多数锁。一旦因为出现问题导致它丢失了大部分锁，协调者就会恢复到一个默认保守状态

除了可用性问题，对于协调者的读写协议必须满足一系列的竞争条件

2.5 分布式存储系统Megastore

在实际的应用中，以下因素能够减轻使用协调者所带来的问题：

- (1)协调者比任何的Bigtable服务器号都简单，基本上没有依赖，所以可用性更高。
- (2)协调者简单、均匀的工作负载让它们能够低成本地进行预防措施。
- (3)协调者轻量的网络传输允许使用高可用连接进行服务质量监控。
- (4)操作者能够在维护期或者故障期集中地让一批协调者失效。当出现某些系统默认的监控信号时这一过程会自动进行。
- (5)Chubby锁的quorum机制能够监测到大多数网络问题和节点的不可用。

2.5 分布式存储系统Megastore

2.5.1 设计目标及方案选择

2.5.2 Megastore数据模型

2.5.3 Megastore中的事务及并发控制

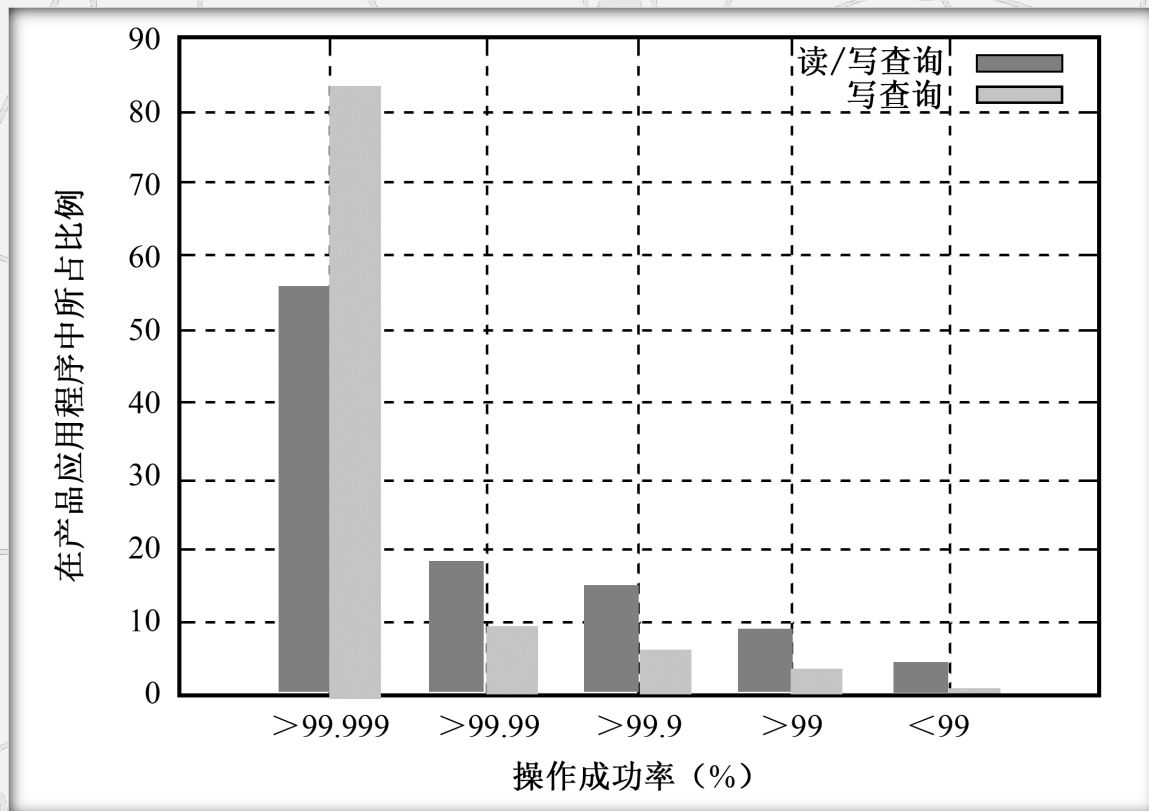
2.5.4 Megastore基本架构

2.5.5 核心技术——复制

▶ 2.5.6 产品性能及控制措施

2.5 分布式存储系统Megastore

• 可用性的分布情况

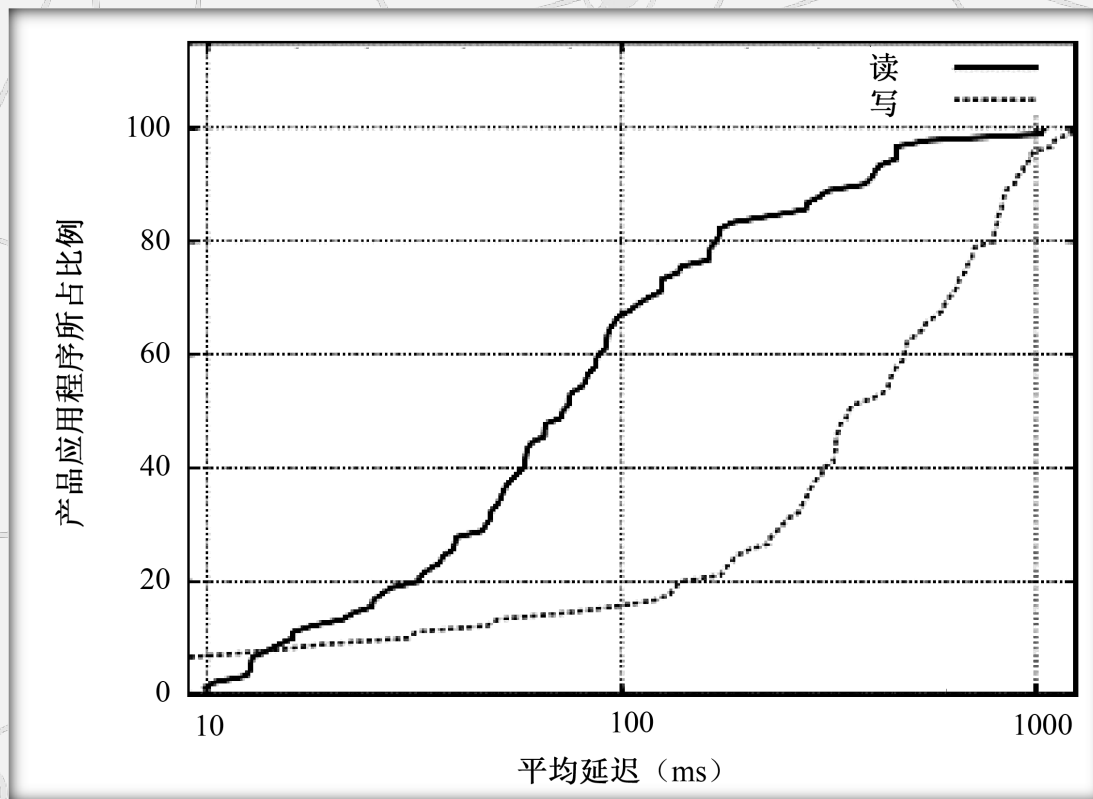


Megastore在Google中已经部署和使用了若干年，有超过100个产品使用Megastore作为其存储系统

从图中可以看出，绝大多数产品具有极高的可用性（>99.999%）。这表明Megastore系统的设计是非常成功的，基本达到了预期目标

2.5 分布式存储系统Megastore

● 产品延迟情况的分布



应用程序的平均读取延迟在**万分之一毫秒**之内，平均写入延迟在100至400毫秒之间

避免Megastore的性能下降，可采取以下三种应对方法：

- (1) 重新选择路由使客户端绕开出现问题的副本
- (2) 将出现问题的副本上的协调者禁用，确保问题的影响降至最小。
- (3) 禁用整个副本

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.6 大规模分布式系统的监控 基础架构Dapper

- ▶ 2.6.1 基本设计目标
- 2.6.2 Dapper监控系统简介
- 2.6.3 关键性技术
- 2.6.4 常用Dapper工具
- 2.6.5 Dapper使用经验

用户的平均每一次前台搜索会导致Google的后台发生

1011次的处理

在我们看来很简单的一次搜索实际上涉及了众多Google后台子系统，
这些子系统的运行状态都需要进行监控

2.6 大规模分布式系统的监控基础架构Dapper

● 两个基本要求



监控系统设计
两个基本要求

1. 广泛可部署性 (Ubiquitous Deployment)

设计出的监控系统应当能够对尽可能多的
Google服务进行监控

2. 不间断的监控

Google的服务是全天候的，如果不能对Google 的后
台同样进行全天候的监控很可能会错过某些无法再现的
关键性故障

2.6 大规模分布式系统的监控基础架构Dapper

• 三个基本设计目标

低开销

这个是广泛可部署性的必然要求。监控系统的开销越低，对于原系统的影响就越小，系统的开发人员也就越愿意接受这个监控系统。

对应用层透明

监控系统对程序员应当是不可见的。如果监控系统的使用需要程序开发人员对其底层的一些细节进行调整才能正常工作的话，这个监控系统肯定不是一个完善的监控系统。

可扩展性

Google的服务增长速度是惊人的，设计出的系统至少在未来几年里要能够满足Google服务和集群的需求。

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

► 2.6.2 Dapper监控系统简介

2.6.3 关键性技术

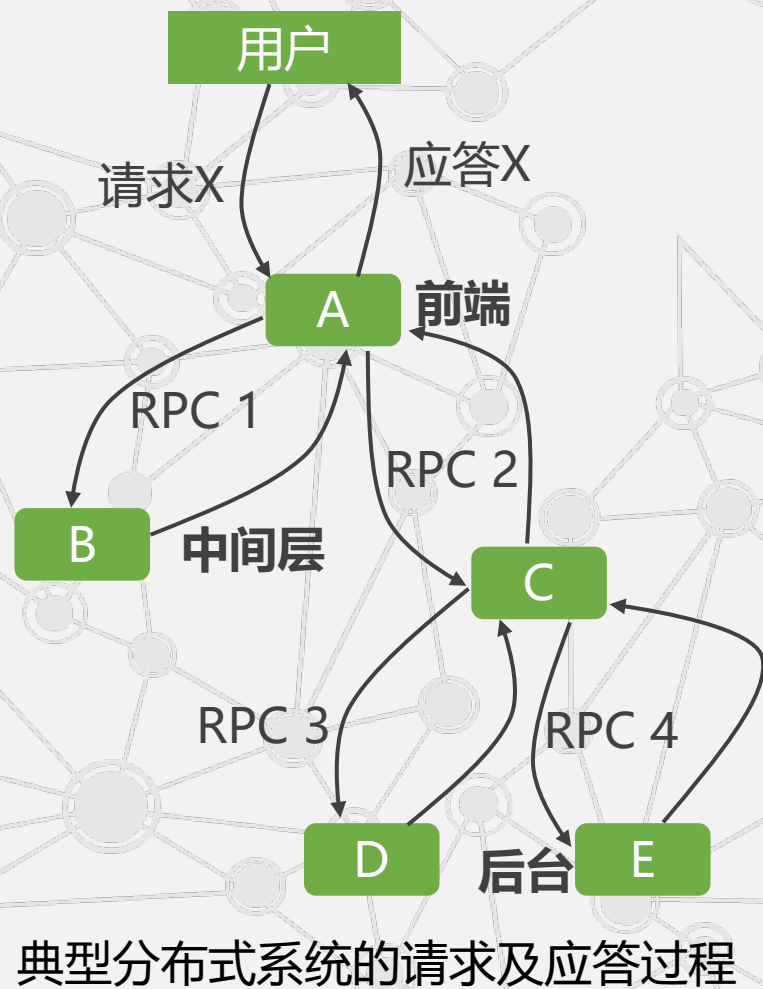
2.6.4 常用Dapper工具

2.6.5 Dapper使用经验

2.6 大规模分布式系统的监控基础架构Dapper

● Dapper监控系统的基本概念

- 在监控系统中记录下所有这些消息不难，如何将这个消息记录同特定的请求（本例中的X）关联起来才是分布式监控系统设计中需要解决的关键性问题之一。



2.6 大规模分布式系统的监控基础架构Dapper

• Dapper监控系统的基本概念

一般来说，有两种方案可供选择：

黑盒(Black Box)方案及基于注释的监控(Annotation-based Monitoring)方案。

二者比较而言，黑盒方案比较轻便，但是在消息关系判断的过程中，黑盒方案主要是利用一些统计学的知识只来进行推断，有时不是很准确。基于注释的方案利用应用程序或中间件给每条记录赋予，一个全局性的标示符，借此将相关消息串联起来。考虑到实际的需求，Google的工程师最终选择了基于注释的方案，为了尽可能消除监控系统的应用程序对被监控系统的性能产生的不良影响，Google的工程师设计并实现了一套轻量级的核心功能库，这将在后面进行介绍。

2.6 大规模分布式系统的监控基础架构Dapper

• Dapper监控系统的三个基本概念

监控树 (Trace Tree)

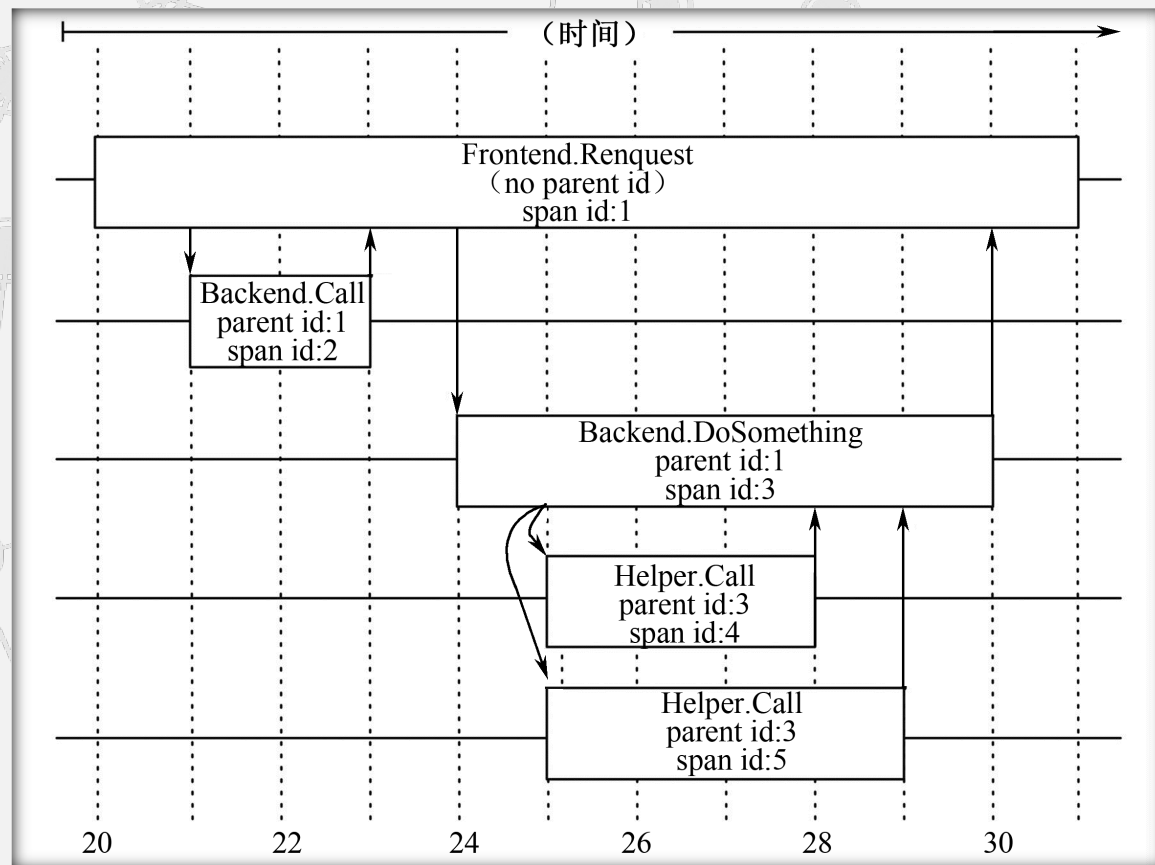
一个同特定事件相关的所有消息

区间 (Span)

区间实际上就是一条记录

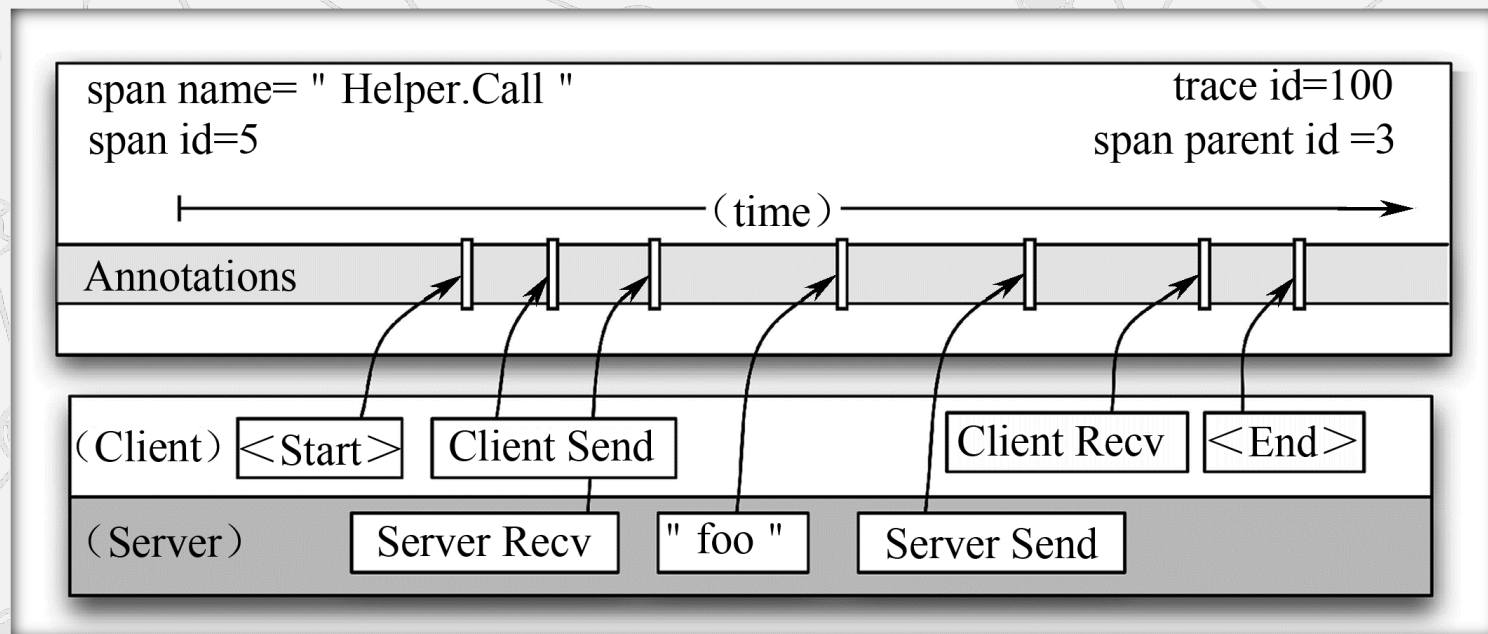
注释 (Annotation)

注释主要用来辅助推断区间关系，也可以包含一些自定义的内容



2.6 大规模分布式系统的监控基础架构Dapper

- 区间Helper.Call的详细信息



区间包含了来自客户端的注释信息：“<Start>”、“Client Send”、“Client Recv”和“<End>”，也包含了来自服务器端的注释信息：“Server Recv”、“foo”和“Server Send”

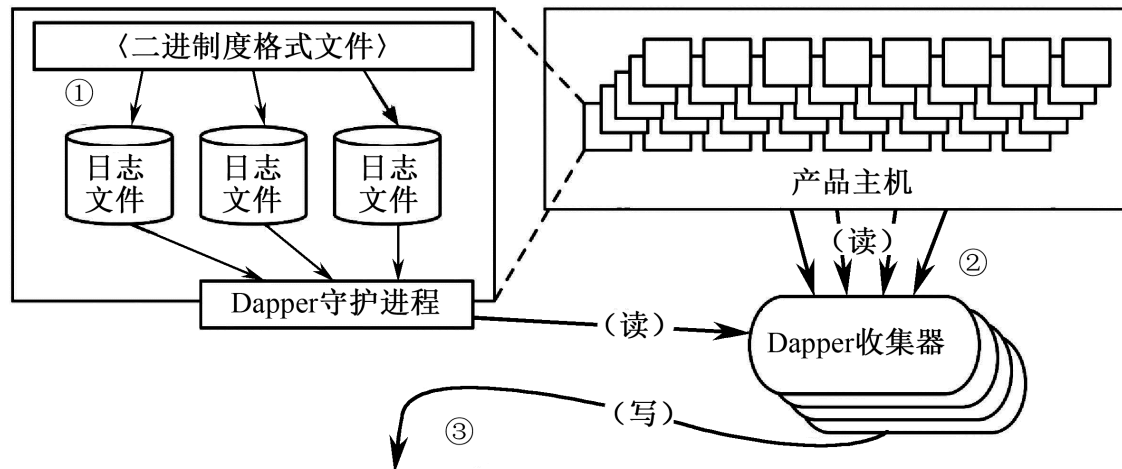
2.6 大规模分布式系统的监控基础架构Dapper

● 监控信息的汇总

(1) 将区间的数据写入到本地的日志文件

(2) 所有机器上的本地日志文件汇集

(3) 汇集后的数据写入到Bigtable存储库中



监控id	区间12	区间23	区间34	区间45	区间56	...
123456	nil	nil	〈数据〉	〈数据〉	nil	...
246802	〈数据〉	nil	nil	nil	〈数据〉	...
357913	nil	〈数据〉	nil	nil	nil	...
...	...	...	...	...	...	...

Bigtable

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

2.6.2 Dapper监控系统简介

► 2.6.3 关键性技术

2.6.4 常用Dapper工具

2.6.5 Dapper使用经验

2.6 大规模分布式系统的监控基础架构Dapper

- 轻量级核心功能库

小规模库

通用线程

(Ubiquitous Threading)

控制流

(Control Flow)

RPC代码库

(RPC Library Code)

- **最关键的代码基础**是基本RPC、线程和控制流函数库的实现
- **主要功能**是实现区间创建、抽样和在本地磁盘上记录日志。
- 将复杂的功能实现限制在一个**轻量级的核心功能库**中保证了Dapper的监控过程基本对应用层透明。

2.6 大规模分布式系统的监控基础架构Dapper

● 二次抽样技术

利用二次抽样技术成功地解决了**低开销及广泛可部署性**的问题。

第一次 抽样

实践中，设计人员发现当抽样率低至 $1/1024$ 时也能够产生足够多的有效监控数据，即在1024个请求中抽取1个进行监控也是可行的，从而可以捕获有效数据

第二次 抽样

发生在数据写入Bigtable前，具体方法是将监控id散列成一个标量 z ($0 \leq z \leq 1$)，如果某个区间的 z 小于事先定义好的汇总抽样系数，则保留这个区间并将它写入Bigtable，否则丢弃

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

2.6.2 Dapper监控系统简介

2.6.3 关键性技术

► 2.6.4 常用Dapper工具

2.6.5 Dapper使用经验

2.6 大规模分布式系统的监控基础架构Dapper

• Dapper存储API

Dapper的“存储API”简称为 **DAPI**，提供了对分散在区域Dapper存储库（DEPOTS）的监控记录的直接访问。一般有以下三种方式访问这些记录。

(1) 通过监控id访问 (Access by Trace id)

利用全局唯一的监控id直接访问所需的监控数据

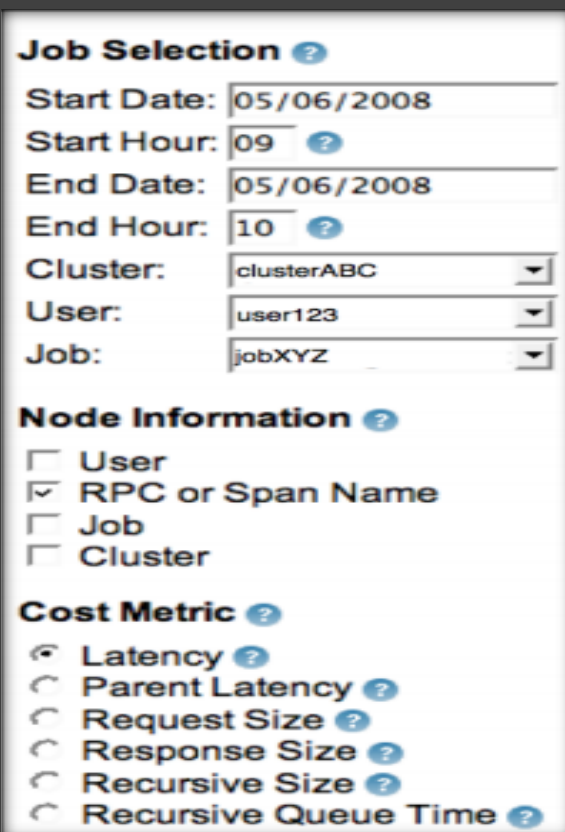
(2) 块访问 (Bulk Access)

借助MapReduce对数以十亿计的Dapper监控数据的并行访问

(3) 索引访问 (Indexed Access)

Dapper存储库支持单索引 (Single Index)

• Dapper用户界面



The screenshot shows the Dapper user interface for job selection and node information. It includes fields for Start Date, Start Hour, End Date, End Hour, Cluster, User, and Job. Below these are checkboxes for Node Information (User, RPC or Span Name, Job, Cluster) and a list of Cost Metrics (Latency, Parent Latency, Request Size, Response Size, Recursive Size, Recursive Queue Time).

Job Selection ?

Start Date: 05/06/2008

Start Hour: 09 ?

End Date: 05/06/2008

End Hour: 10 ?

Cluster: clusterABC

User: user123

Job: jobXYZ

Node Information ?

☐ User

☒ RPC or Span Name

☐ Job

☐ Cluster

Cost Metric ?

☒ Latency ?

☐ Parent Latency ?

☐ Request Size ?

☐ Response Size ?

☐ Recursive Size ?

☐ Recursive Queue Time ?

(1) 选择监控对象

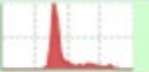
监控的起始时间

区分监控模式的信息

衡量开销的标准

- Dapper用户界面

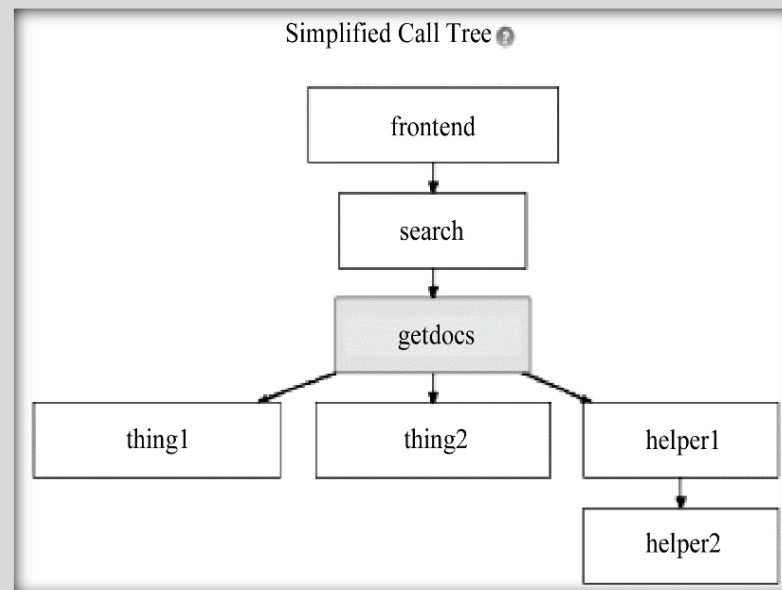
(2) 用户对这些执行模式进行排序并选择查看更多细节

Id ?	Calls ?	Total (ms) ?	Global 90%ile Contribution (count) ? ▾	Local 90%ile (ms) ?	Absolute Histogram (ms) ?	Scaled Histogram (ms) ?	View ?
All ?	40,990,720 (100.00%)	139,773,132.8 (100.00%)	4,098,118 (100.00%)	8.91			View
E	3,450,880 (8.42%)	39,437,312.0 (28.22%)	1,918,437 (46.81%)	19.17			View
R	1,658,880 (4.05%)	55,939,686.4 (40.02%)	1,658,880 (40.48%)	47.21			View

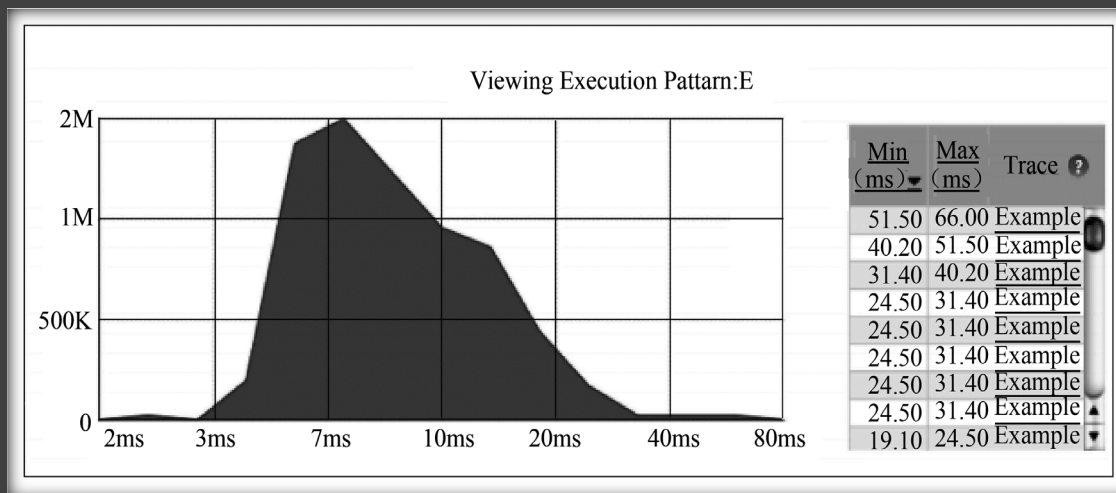
一个大的性能表给出了所有同指定监控对象有关的分布式执行模式的简要情况。

- Dapper用户界面

(3) 分布式执行模式
图形化描述呈现给用户

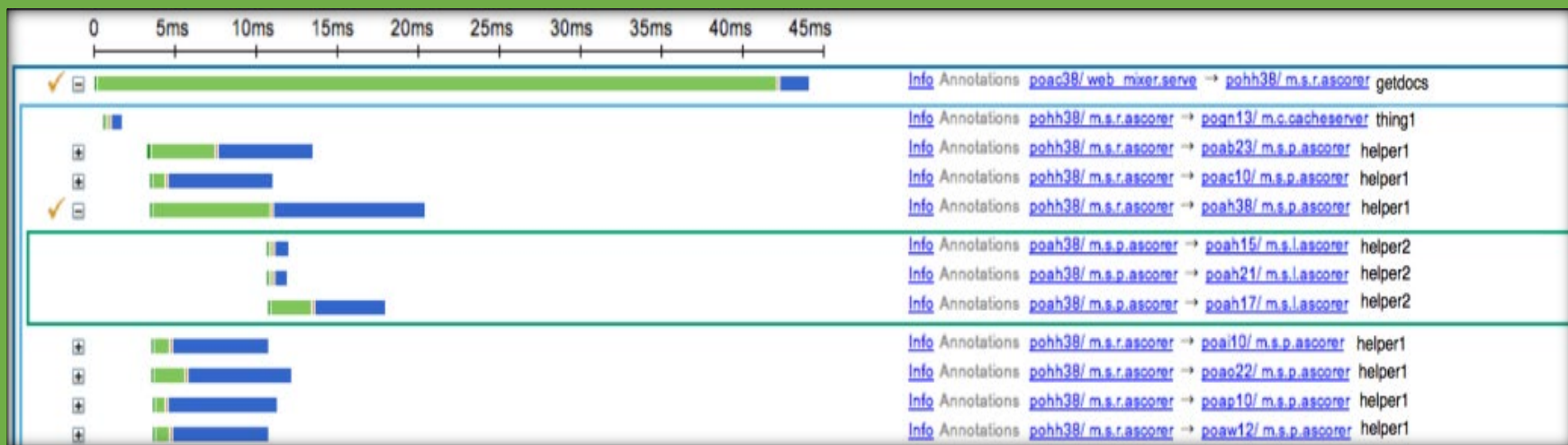


• Dapper用户界面



(4) 根据最初选择的开销度量标准，Dapper以频度直方图的形式将步骤（3）中选中的执行模式的开销分布展示出来，同时还有一系列特殊的监控样例信息，用户可以进一步选择这些监控样例。

• Dapper用户界面



(5) 用户选择了某个监控样例后，就会进入所谓的监控审查视图
(Trace Inspection View)

顶端-全局的时间线

2.6 大规模分布式系统的监控 基础架构Dapper

2.6.1 基本设计目标

2.6.2 Dapper监控系统简介

2.6.3 关键性技术

2.6.4 常用Dapper工具

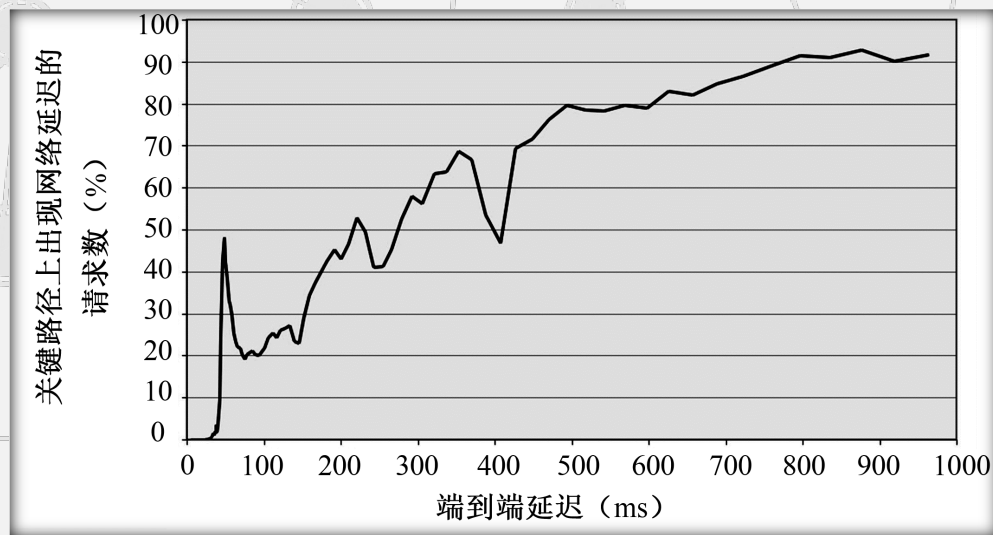
▶ 2.6.5 Dapper使用经验

2.6 大规模分布式系统的监控基础架构Dapper

• Dapper使用经验

1 新服务部署中Dapper的使用
利用Dapper对系统延迟情况进行一系列的跟踪，进而发现存在的问题

2 定位长尾延迟 (Addressing Long Tail Latency)
端到端性能和关键路径上的网络延迟有着极大的关系



2.6 大规模分布式系统的监控基础架构Dapper

• Dapper使用经验

3

推断服务间的依存关系 (Inferring Service Dependencies)

Google的“服务依存关系”项目使用监控注释和DPAI的MapReduce接口实现了服务依存关系确定的自动化

4

确定不同服务的网络使用情况

利用Dapper平台构建了一个连续不断更新的控制台，用来显示内部集群网络通信中最活跃的应用层终端

5

分层的共享式存储系统

没有Dapper之类的工具的情况下对于这种共享式服务资源的争用也同样难以调试

6

利用Dapper进行“火拼” (Firefighting with Dapper)

Dapper用户可以通过和Dapper守护进程的直接通信，将所需的最新数据汇总在一起



本章未完待续



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

谢谢观看